

DISTRIBUTED TRAINING

What Actually Breaks When You Scale Training Across Multiple GPUs

Duplicate data across GPUs, misplaced gradient
accumulation,
and BF16 underflow that never throw an exception.

Dibayendu Mukherjee

AI Research Engineer · Systems Engineer

The Setup

Scaling from prototype to production uncovered issues that single-GPU tests did not surface.

What I Was Building

- A transformer model exceeding single-GPU memory limits to improve task accuracy and enable larger context windows.
- Multi-GPU training using PyTorch DistributedDataParallel (DDP).
- Mixed precision (FP16/BF16) to reduce memory and speed up training.
- Gradient accumulation to simulate larger global batch sizes without extra GPU memory.

Why I scaled up: to train a larger model for better accuracy and to shorten experiment turnaround time.

The training loop ran on one GPU, but scaling to 4 GPUs introduced subtle, silent failures not seen during single-GPU prototyping.

Key takeaway:

Scaling hardware changes numerical and synchronization behavior — more GPUs do not equal faster results without additional changes.

What Actually Went Wrong

The code ran, the loss decreased, but after 2k steps the validation loss stalled at 3.1 while the single-GPU baseline hit 2.7. No errors, no NaNs. Three silent bugs caused it.

Bug #1: Every GPU Saw the Same Data

I forgot to wrap the dataset in `DistributedSampler`. All 4 GPUs received identical batches, so the model effectively trained on $\frac{1}{4}$ of the intended data diversity per step. This caused overfitting to a narrower distribution and degraded generalization.

Bug #2: Gradient Accumulation Done Wrong

I called `optimizer.step()` inside the accumulation loop instead of after 16 micro-batches. This:

- Triggered gradient all-reduce 16 times per optimizer step, so communication overhead reduced throughput (observed $2.1\times$ vs expected $3.8\times$).
- The model never saw the intended global batch of 512; each GPU used batch size 8. I had scaled the learning rate for batch 512 (base LR $\times 64$), but because I stepped every micro-batch, the actual global batch seen by the optimizer was only 32 (4 GPUs $\times$ 8). The correct LR for batch 32 would have been base $\times 4$, so my LR was $16\times$ too large, causing unstable updates and a plateaued loss.

Bug #3: BF16 Underflow without GradScaler

Using BF16 autocast, some gradients (especially in the attention output projection) became so small that the 7-bit mantissa rounded them to zero. These layers stopped learning entirely. No exception was raised; the optimizer simply skipped updating them. I only noticed by printing per-layer gradient norms.

What I Did (Wrong)

- Forgot `DistributedSampler` → duplicate data across GPUs.
- Called `optimizer.step()` every micro-batch → broke gradient accumulation and scaled LR incorrectly.
- Used BF16 without `GradScaler` → silent gradient underflow.
- Assumed DDP + mixed precision would handle everything automatically.

What I Fixed

- Added `DistributedSampler` with `shuffle=True` and per-epoch seed.
- Moved `optimizer.step()` outside the accumulation loop; zeroed gradients once per optimizer step.
- For FP16, used `torch.cuda.amp.GradScaler`. For BF16, monitored gradient norms and cast sensitive ops (attention output projection) to FP32 to avoid rounding to zero.
- Used linear LR scaling with warmup: `lr = base_lr * 64`, then 500-step warmup, and gradient clipping at 1.0.

Outcome After Fixes

Validation loss dropped from **3.1** → **2.72** (within noise of single-GPU 2.7). Throughput improved from **2.1×** → **3.5×** (88% of ideal scaling). No crashes, but the difference was night and day — and all three bugs would have been invisible to standard monitoring.

What I Saw: Three Silent Failures

No crashes. No exceptions. Just three mistakes that quietly wasted 2,000 GPU-hours and produced a worse model.

1. Every GPU Trained on the Same Data

Duplicate batches.

Punchline: Forgot `DistributedSampler`, so all 4 GPUs saw identical mini-batches. The model saw only $\frac{1}{4}$ of the data diversity per step and overfit.

Symptom: Validation loss plateaued at 3.1 vs single-GPU 2.7 after 2k steps.

Human oversight: Always wrap the dataset in `DistributedSampler` with `shuffle=True`.

2. Gradient Accumulation Called in the Wrong Place

Extra syncs.

Punchline: Called `optimizer.step()` inside the accumulation loop, causing 16 gradient all-reduces per optimizer step instead of one. Throughput collapsed from 3.8× to 2.1×.

Symptom: Communication overhead dominated, and the model never actually trained with global batch 512 — only per-GPU batch 8.

Human oversight: Place `optimizer.step()` after the accumulation loop, and zero gradients once per optimizer step.

3. Mixed Precision Silently Killed Gradients

Precision loss.

Punchline: BF16 autocast rounded small gradients in attention projection layers to zero. Those layers stopped learning, but no error was raised.

Symptom: Per-layer gradient norms printed as exactly 0.0 after step 500.

Human oversight: Use `GradScaler` for FP16, or monitor gradient norms and cast sensitive ops to FP32 for BF16.

The Real Danger

Distributed training doesn't break loudly — it silently wastes compute and produces inferior models. Only human intuition, explicit checks, and rigorous process controls can catch these failures before they burn your budget.

Why It Happened

Three specific oversights compounded into a silently degraded model. None of them raised an exception.

Technical Causes

- **Forgot DistributedSampler** → all GPUs received identical batches, reducing effective data diversity by 4×.
- **Called `optimizer.step()` inside the accumulation loop** → 16 gradient all-reduces per optimizer step instead of one, plus the model never saw global batch 512.
- **Used BF16 without GradScaler** → gradients in attention projection underflowed to zero, silently freezing those layers.

Process Causes

- **No per-layer gradient monitoring** → zeroed gradients went unnoticed until loss plateaued.
- **No throughput benchmark** → 2.1× scaling was accepted without questioning the missing 3.8× target.
- **No distributed dry-run** → data distribution and accumulation logic were never validated on 2 GPUs before scaling to 4.
- **Assumed DDP + mixed precision handle everything** → skipped reading the documentation on sampler and scaler requirements.

How I Fixed It

Three targeted fixes addressed the exact bugs — no generic checklist, just the specific changes that recovered performance.

1. Added **DistributedSampler** to the **DataLoader**

Each GPU now receives a unique, shuffled subset of the dataset per epoch. This restored full data diversity per global step and eliminated the overfitting caused by duplicate batches.

2. Moved **optimizer.step()** Outside the **Accumulation Loop**

Gradients now accumulate across 16 micro-batches before a single all-reduce and optimizer step. This reduced communication overhead by 16× and let the model actually train with global batch 512.

3. Added **GradScaler** and **Gradient Norm Monitoring**

For FP16, **GradScaler** prevents small gradients from underflowing. For BF16, I monitored per-layer gradient norms and cast sensitive ops (attention output projection) to FP32 to prevent rounding to zero.

How I Implemented It

These code snippets show the exact fixes for the three silent bugs.

1. DistributedSampler + DataLoader (Python)

```
from torch.utils.data import DataLoader, DistributedSampler

sampler = DistributedSampler(
    dataset,
    num_replicas=world_size,
    rank=local_rank,
    shuffle=True,
    seed=42
)

loader = DataLoader(
    dataset,
    batch_size=8,
    sampler=sampler,
    num_workers=4,
    pin_memory=True
)

# Call this at the start of every epoch
sampler.set_epoch(epoch)
```

2. Correct Gradient Accumulation Loop for BF16 (no scaler)

```
# Correct gradient accumulation loop (BF16, no scaler needed)
optimizer.zero_grad()
for step, batch in enumerate(loader):
    with torch.cuda.amp.autocast(dtype=torch.bfloat16):
        loss = model(batch)
        loss = loss / 16 # Normalize for accumulation

    loss.backward() # No scaler for BF16

    if (step + 1) % 16 == 0:
        optimizer.step()
        optimizer.zero_grad()
```

3. FP16 with GradScaler (and gradient monitoring)

```
# If using FP16 instead of BF16, use GradScaler
scaler = torch.cuda.amp.GradScaler()

optimizer.zero_grad()
for step, batch in enumerate(loader):
    with torch.cuda.amp.autocast(dtype=torch.float16):
        loss = model(batch)
        loss = loss / 16

    scaler.scale(loss).backward()

    if (step + 1) % 16 == 0:
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad()

# Monitor gradient norms (works for both BF16 and FP16)
for name, param in model.named_parameters():
    if param.grad is not None:
        grad_norm = param.grad.norm().item()
        if grad_norm == 0.0:
            print(f"WARNING: Zero gradient at {name}")
```

THE MLOPS TAKEAWAY

Scale the system, not just the model.

Multi-GPU training isn't a hardware upgrade — it's a systems engineering problem. Gradient sync, memory, and learning rates all interact in ways that silently degrade training.

If you don't think in systems, more GPUs just mean more expensive failures.

What's the most surprising thing that broke when you scaled your training?

Building AI systems by understanding the mathematics, engineering, and intuition behind every architecture.

GitHub: github.com/MrHeaven1y
Portfolio: mrheaven1y.github.io
LinkedIn: linkedin.com/in/dibayendu-mukherjee-bb897b267

Dibayendu Mukherjee

THE SERIES JOURNEY

From Notebook to Distributed Systems

POST 1: PREPROCESSING PARITY

"Silent failures happen when training and serving differ."

Data leakage and pipeline mismatches between training and inference ruin real-world accuracy.

POST 2: VALIDATION DRIFT & EVALUATION GAPS

"Great metrics mean nothing if your validation set lies."

Random splits, wrong business metrics, and unmonitored drift create an illusion of model quality.

POST 3: MULTI-GPU TRAINING FAILURES

"More GPUs make failures quieter, not louder."

Duplicate data across GPUs, misplaced gradient accumulation, and BF16 underflow silently degrade training.

The pattern across all three: AI doesn't crash — it convinces you it's right.
Human guardrails at every stage are the only defense.

Dibayendu Mukherjee

AI Research Engineer · Systems Engineer

Thanks for following the series.

If these deep dives resonated, let's connect
and build better systems together.

GitHub: github.com/MrHeavenly

Portfolio: mrheavenly.github.io

LinkedIn: linkedin.com/in/dibayendu-mukherjee-bb897b267

Dibayendu Mukherjee

AI Research Engineer & Systems Engineer